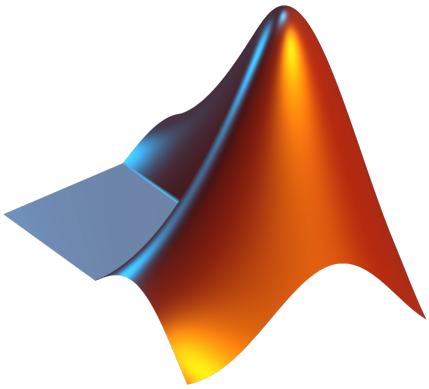
Matrices en MATLAB y Respuesta Transitoria

Fabrizio Benavidez[1]

[1] *Facultad de Ciencias Exactas, Físicas y Naturales, Ingeniería Aeronáutica*

Resumen. El siguiente trabajo práctico representa la manipulación y creación de matrices y vectores usados en MATLAB. Además, se utiliza los ciclos para la generación de gráficas en un `subplot`, así como la función `zpk` para generar funciones de transferencias en base a sus ganancias, ceros y polos. Este último fue necesario para el análisis en la respuesta transitoria para distintas configuraciones de un sistema de segundo orden.

Palabras claves: ciclos `for`, `subplot`, `zpk`, respuesta transitoria.



12 de septiembre de 2026

Semana 4

[1] `fabrizio.benavidez@mi.unc.edu.ar`

Guía 07

La siguiente tabla representa la consigna, código MATLAB y salida para la guía 07.

Consigna	**Código MATLAB**	**Salida**	**Dimensión**
v_1 (vector fila)	`v1 = [1 2 3];`	`1 2 3`	1×3

v_2 (vector columna)	`v2 = [4; 5; 6];`	`4`
		`5`
		`6`
			3×1

M_1 (matriz)	`M1 = [1 1 0; 3 0 7; 5 4 2];`	`1 1 0`
		`3 0 7`
		`5 4 2`
			3×3

M_2 (matriz)	`M2 = [0 2 3; -1 0 5; 0 -6 9];`	`0 2 3`
		`-1 0 5`
		`0 -6 9`
			3×3

$M_1 \cdot M_2$	`M1*M2`	`-1 2 8`
		`0 -36 72`
		`-4 -2 53`
			3×3

$M_1 \cdot v_2$	`M1*v2`	`9`
		`54`
		`52`
			3×1

$M_1 \cdot v_1$	`M1*v1`	`Incorrect dimensions for matrix multiplication.`	–

$M_1 \cdot v_1^T$	`M1*v1'`	`3`
		`24`
		`19`
			3×1

$v_1 \cdot M_2$	`v1*M2`	`-2 -16 40`	1×3
$v_1 \cdot v_2$	`v1*v2`	`32`	1×1
$v_1 \cdot v_2^T$	`v1*v2'`	`Incorrect dimensions for matrix multiplication.`	–
$v_1 \odot v_2^T$	`v1.*v2'`	`4 10 18`	1×3

$M_1 \odot M_2$	`M1.*M2`	`0 2 0`
		`-3 0 35`
		`0 -24 18`
			3×3

Elemento $(2,3)$ de M_1	`M1(2,3)`	`7`	1×1
Fila 3 de M_1	`M1(3,:)`	`5 4 2`	1×3

Reemplazar ceros de M_1 por 9	`M1(M1 == 0) = 9;`	`1 1 9`
		`3 9 7`
		`5 4 2`
			3×3

$M_2(:,2) \cdot v_1$	`M2(:,2)*v1`	`2 4 6`
		`0 0 0`
		`-6 -12 -18`
			3×3

$M_2(1,:) \cdot v_2$	`M2(1,:)*v2`	`28`	1×1
$v_1(\text{end})$	`v1(end)`	`3`	1×1
$v_2(\text{end}-1)$	`v2(end-1)`	`5`	1×1
$M_2(\text{end})$	`M2(end)`	`9`	1×1

$M_1(:,\text{end})$	`M1(:,end)`	`0`
		`7`
		`2`
			3×1

Continúa en la siguiente página...

Consigna	**Código MATLAB**	**Salida**	**Dimensión**

$M_1(:, \text{end}-1)$	`M1(:,end-1)`	`1`
		`0`
		`4`
			3×1

Eliminar último elem. v_1	`v1(end) = [];`	`1 2`	1×2

Eliminar columna 2 de M_2	`M2(:,2) = [];`	` 0 3`
		`-1 5`
		` 0 9`
			3×2

$M_1 \cdot 6$	`M1*6`	` 6  6  0`
		`18  0 42`
		`30 24 12`
			3×3

$M_1/3$	`M1/3`	`0.3333 0.3333 0.0000`
		`1.0000 0.0000 2.3333`
		`1.6667 1.3333 0.6667`
			3×3

$M_1 + 2$	`M1+2`	`3 3 2`
		`5 2 9`
		`7 6 4`
			3×3

Autovalores de M_1	`eig(M1)`	` 6.3149`
		`-3.3496`
		`-0.0347`
			3×1

Autovectores y Autovalores	`[V,D] = eig(M1);`	`V (Matriz de autovectores)` `D (Matriz diagonal)`	$V : 3 \times 3$ $D : 3 \times 3$
Determinante de M_1	`det(M1)`	`1`	1×1
Dimensiones de M_1 y M_2	`size(M1), size(M2)`	`3 3`	1×2
Cant. de filas de v_1	`size(v1,1)`	`1`	1×1
Cant. de columnas de v_1	`size(v1,2)`	`3`	1×1
Longitud de v_1	`length(v1)`	`3`	1×1
Longitud de v_2	`length(v2)`	`3`	1×1
Máximo por columna (M_1)	`max(M1)`	`5 4 7`	1×3
Valor máximo global (M_1)	`max(M1(:))`	`7`	1×1

En el siguiente cuadro se muestra cómo trabajar con matrices y generar distintas formas matriciales:

Consigna	**Código MATLAB**	**Salida**	**Dimensión**

Matriz de coeficientes A	`A = [2 log(10) -1;`
	`1 -5 -pi; sqrt(2) -1`
	`7];`
		`2.0000  2.3026 -1.0000`
		`1.0000 -5.0000 -3.1416`
		`1.4142 -1.0000  7.0000`
			3×3

Solución mediante $x = A^{-1}b$	`x = inv(A)*[2; 4; 6]`	` 1.9333`
		`-0.6483`
		` 0.3739`
			3×1

Solución mediante operador \	`x = A\[2; 4; 6]`	` 1.9333`
		`-0.6483`
		` 0.3739`
			3×1

Matriz de ceros	`zeros(3,4)`	Matriz de ceros	3×4
Matriz de unos	`ones(7,4)`	Matriz de unos	7×4

Continúa en la siguiente página...

Consigna	**Código MATLAB**	**Salida**	**Dimensión**
Matriz identidad	`eye(3,3)`	Matriz identidad I_3	3×3
Identidad abreviada	`eye(3)`	Matriz identidad I_3	3×3
Vector columna de `NaN`	`nan(4,1)`	Vector de `NaN`	4×1
Vector fila de `NaN`	`nan(1,4)`	Vector de `NaN`	1×4
Cuadrado mágico 5×5	`M = magic(5)`	17 24 1 8 15 23 5 7 14 16 4 6 13 20 22 10 12 19 21 3 11 18 25 2 9	5×5
Suma de la 1.ª columna de M	`sum(M(:,1))`	65	1×1
Suma de la 2.ª fila de M	`sum(M(2,:))`	65	1×1
Distribución uniforme $[0, 1]$	`rand(2,6)`	Valores aleatorios en $(0, 1)$	2×6
Escalar aleatorio $[0, 1]$	`rand`	Valor aleatorio en $(0, 1)$	1×1
Uniforme en rango $[10, 15]$	`10 + 5*rand(2,6)`	Valores aleatorios en $(10, 15)$	2×6
Normal estándar $(\mu = 0, \sigma^2 = 1)$	`randn(4,2)`	Valores aleatorios Gaussianos	4×2
Matriz normal 3×3	`randn(3)`	Valores aleatorios Gaussianos	3×3

Por último, generemos un vector de 5000 elementos según la distribución gaussiana y generemos un histograma.

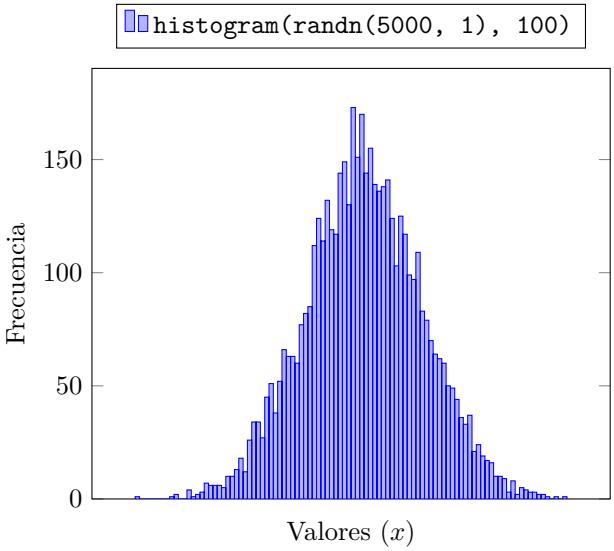


Figura 1. Distribución Gaussiana discreta ($\mu = 0,\ \sigma^2 = 1$)

Notar que `magic()` genera un cuadrado mágico, esto es, cada fila y columna (incluido la diagonal principal) suman el mismo número.

Guía 08

Definamos la siguiente función transferencia:

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \tag{1}$$

Si $\omega_n = 3\,\text{rad/s}$, definiremos distintos factores de amortiguamiento para ver el comportamiento del sistema. En la Figura 2 se muestran las respuestas del sistema para factores de amortiguamiento iguales a $[3, 0{,}6, -0{,}4, -2]$.

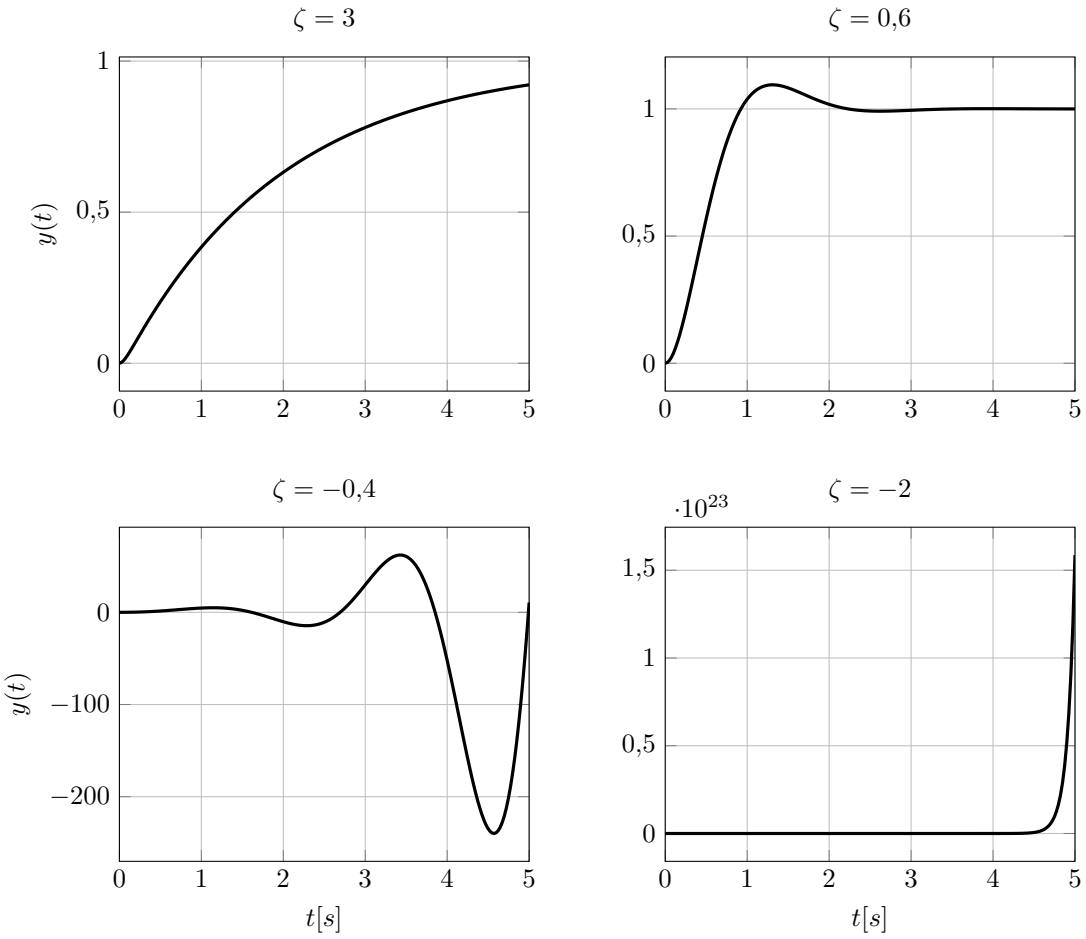


Figura 2. Sistema de segundo orden con factores de amortiguamiento variables. Si $0 < \zeta < 1$, entonces el sistema es **subamortiguado** y presentará algún sobrepaso o sobreenlongación. Si $\zeta > 1$, el sistema es **sobreamortiguado** y no presentará sobrepaso. Si $\zeta = 0$, entonces el sistema es oscilante y nunca se amortiguará. Si $\zeta < 0$, el sistema está en un equilibrio inestable y diverge (no tiene sentido hablar sobre factores de amortiguamiento negativos).

Una forma de verificar más rápido sobre la estabilidad del sistema es usar la función `pole()` de MATLAB. En la Tabla 3 se observa que, cuando $\zeta > 0$, la parte real de los polos es negativa, lo cual significa que la estabilidad del sistema es positiva (no diverge). En cambio, cuando la parte real de los polos son positivos el sistema diverge.

Factor de amortiguamiento ζ	Polos del sistema
3	$p_1 = -17{,}4853$, $p_2 = -0{,}5147$
0.6	$p_{1,2} = -1{,}8 \pm 2{,}4i$
-0.4	$p_{1,2} = 1{,}2 \pm 2{,}7495i$
-2	$p_1 = 11{,}1962$, $p_2 = 0{,}8038$

Tabla 3: Polos para cada configuración del sistema de segundo orden con frecuencia constante

Por último, definamos las siguientes funciones de transferencias:

$$G_1(s) = \frac{2}{s^2 + 2s + 2} \qquad G_2(s) = \frac{10}{s^2 + 2s + 10} \tag{2}$$

$$G_3(s) = \frac{18}{s^2 + 6s + 18} \qquad G_4(s) = \frac{s + 18}{s^2 + 6s + 18} \tag{3}$$

De la Figura 3 se observa las respuestas del sistema y en la Figura 4 el mapa de los polos y ceros de cada uno de ellos. Como se puede ver, los sistemas G_1, G_3 y G_4 presentan el mismo sobrepaso, esto se debe a que los polos de los respectivos sistemas están sobre el mismo ángulo de 45° (ver Figura 4). Los tiempos picos son distintos debido a la frecuencia natural no amortiguada. El tiempo pico se define como:

$$t_p = \frac{\pi}{\omega_n \sqrt{1 - \zeta^2}} \tag{4}$$

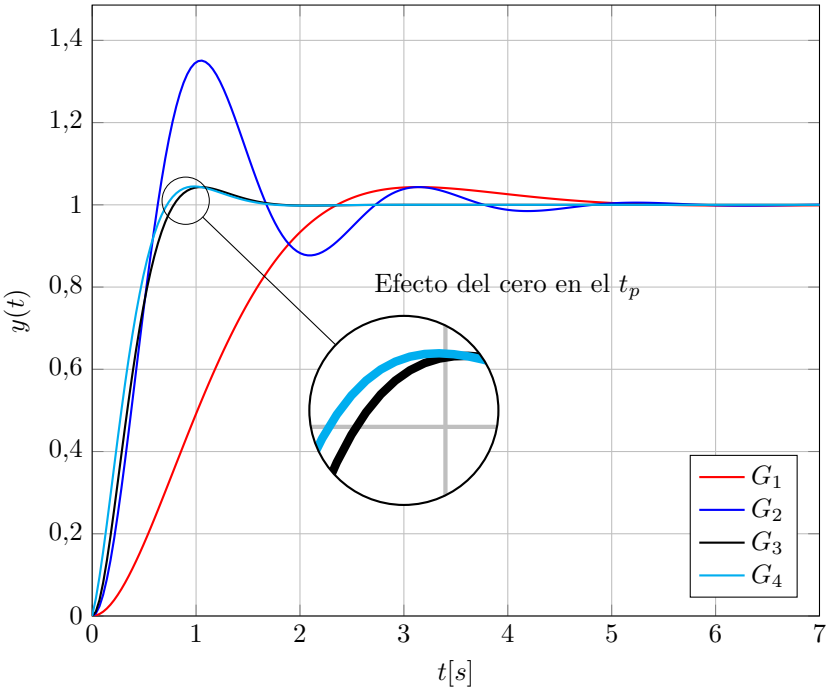


Figura 3. Gráfica de las respuestas de las funciones transferencias.

Tanto G_1, G_3 y G_4 están sobre la misma línea de 45°, entonces su amortiguamiento son $\zeta = 1/\sqrt{2}$, pero su frecuencia natural no son iguales, ya que $\omega_{n_1} = \sqrt{2}$ y $\omega_{n_{3,4}} = 3\sqrt{2}$. Entonces, al calcular los tiempos picos de cada uno de ellos se obtiene:

$$t_{p_1} = \pi \qquad t_{p_{3,4}} = \frac{\pi}{3} \tag{5}$$

Se observa que G_4 presenta un tiempo t_{p_4} menor a t_{p_3}, y esto se debe a que presenta un cero en su sistema, acelerando la respuesta transitoria (ver Figura 3).

Por último, G_2 presenta un sobrepaso más grande debido a que los polos se encuentran a un ángulo θ mayor, por lo tanto ζ es menor.

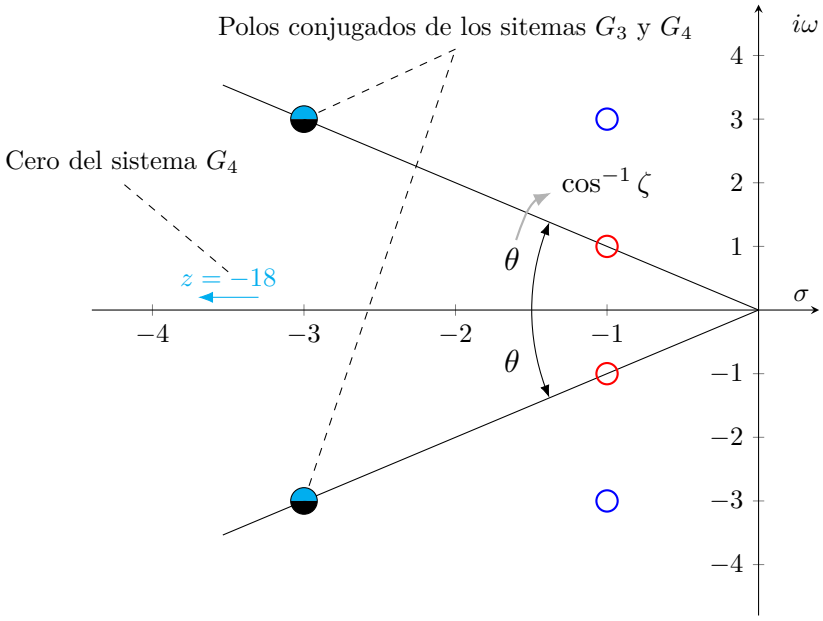


Figura 4. Mapa de los polos y ceros de los sistemas.

A continuación se presentan los códigos MATLAB utilizados en este trabajo práctico

```
clear all;

G1 = zpk([], [-1-i, -1+i], 2);
G2 = zpk([], [-1-3i, -1+3i], 10);
G3 = zpk([], [-3-3i, -3+3i], 18);
G4 = zpk([-18], [-3-3i, -3+3i], 1);

% figure;
% hold on;
% step(G1)
% step(G2)
% step(G3)
% step(G4)

%Verifico el tiempo de sobreenlongaci n
stepinfo(G2)

%%
syms s t

Rs = 1/s;
G1_sym = 2/(s^2 + 2*s + 2);
G2_sym = 10/(s^2 + 2*s + 10);
G3_sym = 18/(s^2 + 6*s + 18);
G4_sym = (s+18)/(s^2 + 6*s + 18);

ilaplace(G4_sym*Rs, s, t)

```

Código 1. Entrada escalón a distintos sistemas de segundo orden con variación en sus polos, ceros y ganancias.

```
% frecuencia natural no amortiguada
w = 3
% %factor de amortiguamiento relativo
% z = 0.4
%Nombre del archivo
mdl = "Curso_Matlab\semana4\mod_08.slx";
t_sim = '5'; %tiempo de ejecuci n
% mod = sim(mdl, "StopTime", t_sim)
for i = 1:4
    switch i
        case 1
            z = 3;
        case 2
            z = 0.6;
        case 3
            z = -0.4;
        case 4
            z = -2;
    end
    %atenuaci n
    o = z*w;
    mod = sim(mdl, 'StopTime', t_sim)
    %funci n transferencia
    num = [w^2];
    den = [1 2*o w^2];
    G = tf(num, den);
    display(G)
    disp(['Polos del sistema', num2str(i)])
    polos_G = pole(G)
    % extraigo los datos del simulink
    data = mod.yout.extractTimetable;
    t = data.Data; x = data.Data_1; y = data.Data_2
    subplot(2, 2, i)
    plot(t, y)
    title("z = " + z)
end
%%
syms s t omega_n zeta
w = 3;
Gs(zeta, omega_n) = omega_n^2 / (s^2 + 2*zeta*omega_n*s + omega_n^2);
Rs = 1/s; %entrada escal n

ilaplace(Gs(-2, w)*Rs, s, t)

```

Código 2. Ciclo `for` utilizado para generar los subplots y verificación de la respuesta en el dominio del tiempo usando cálculo simbólico